2025

TrueRepairs

CSC-4996 SRS DOCUMENT

HISHAM ISSA, ROBERT SIMOVSKI, ISHMAEL GONZALES, JIBRAN IQBAL

Table of Contents

1. Introduction....4
1.1 Overview of the SRS....4
1.2 Purpose....4
1.3 Scope....5
1.4 Definitions, Acronyms, and Abbreviations....6
1.5 References....6
1.6 Overview of the SRS Structure....7
2. General Factors....7
2.1 Product Perspective....7
2.1.1 Context....8
2.1.2 System Perspective....8
2.1.3 Diagram....8
2.2 Product Functions....9
2.2.1 User Registration and Login....9
2.2.2 Repair Cost Estimation....9
2.2.3 AI-Powered Damage Detection....9
2.2.4 Dashboard and Repair History....9
2.2.5 Parts Pricing Transparency....9
2.3 User Characteristics....9
2.3.1 DIY Enthusiasts....9
2.3.2 Young Professionals....10
2.3.3 General Drivers....10
2.4 General Constraints....10
2.4.1 Technical Standards....10
2.4.2 Hardware and Software Limitations....10
2.4.3 Performance Constraints....10
2.5 Assumptions and Dependencies....10
2.5.1 Assumptions....10

2.5.2 Dependencies....11
3. Requirements....11
3.1 External Interface Requirements....11
3.1.1 User Interfaces....11
3.1.2 Hardware Interfaces....12
3.1.3 Software Interfaces....13
3.1.4 Communication Interfaces....13
3.1.5 Functional Requirements....14
3.3 Non-Functional Requirements....17
3.3.1 Performance....17
3.3.2 Reliability....17
3.3.3 Availability....18
3.3.4 Security....18
3.3.5 Maintainability....18
3.3.6 Portability....18
3.4 Design Constraints....18
3.4.1 Hardware Limitations....18
3.4.2 API Dependencies....18
3.4.3 Standards Compliance....18
3.4.4 Client Constraints....19
3.5 Logical Database Requirements....19
3.5.1 Database Usage....19
3.5.2 Types of Information....19
3.5.3 Frequency of Use....19
3.5.4 Accessing Capabilities....19
3.5.5 Data Entities and Relationships....19
3.5.6 Integrity Constraints....19
3.5.7 Data Retention....20
3.6 Other Requirements....20

3.7 Consolidated Error Handling.....20
4. Analysis Models.....21
5. Traceability Matrix 24

1. Introduction

1.1 Overview of the SRS

This Software Requirements Specification (**SRS**) outlines the full set of functional and non-functional requirements for the **TrueRepairs** system. The primary goal of this document is to ensure a shared, verifiable understanding among all stakeholders including developers, testers, and project sponsors "in the future" so that the system can be designed, implemented, and evaluated consistently. It provides a clear description of the system's objectives, required features, operational constraints, and expected user interactions.

1.2 Purpose

The purpose of this SRS is to provide a **complete, unambiguous, and verifiable description** of the TrueRepairs system. By doing so, it ensures that the system fulfills its intended function: helping drivers make informed repair decisions.

This purpose can be broken down by audience:

- **Developers:** The SRS offers technical details that ensure the frontend and backend teams work from the same playbook. For instance, when the backend team implements API calls to OpenAI, they will consult Section 3.1.3 (Software Interfaces) for exact specifications. The frontend team, meanwhile, will rely on Section 3.1.1 (User Interfaces) to design the dashboard and homepage consistently.
- **Quality Assurance Team:** Testers will derive their test plans directly from this SRS. For example, the requirement "FR2: User Sign-In" specifies inputs, outputs, and error handling. QA can translate these directly into test cases for verifying login success, invalid credentials, and password mismatches.
- **Stakeholders (Professor, GTA, Project Reviewers):** The SRS demonstrates that the project is academically rigorous, aligns with course objectives, and integrates multiple technologies. It also shows that the project is realistic and feasible within the semester timeline.
- **End Users (Drivers):** While they may not read the SRS directly, the purpose of the system is rooted in their needs. Everyday drivers — whether budget-conscious college students, parents managing expenses, or DIY enthusiasts — will indirectly benefit because this document guarantees their needs are reflected in the system's functionality.

The ultimate purpose of TrueRepairs is to **empower drivers with transparency**. Whether a user chooses to repair their car at home or at a professional shop, the system provides them with the data to make an informed decision.

1.3 Scope

The scope of the TrueRepairs project defines both the breadth of the system's functionality and its limitations. By setting clear boundaries, the team ensures that the project remains achievable, focused, and valuable.

In-Scope Features:

- **Repair Cost Estimator:** Users can input a VIN, make, model, or year, and the system returns estimated repair costs. Data comes from external cost and pre integrated data linked to valid sources.
- **DIY vs. Professional Comparison:** Each estimate includes both DIY parts costs and professional labor costs, giving users the ability to compare.
- **AI Damage Detection:** Users may upload an image of their car. The system identifies damaged exterior panels and links them to repair costs.
- **User Accounts:** Registration, login, and profile management allow users to store their vehicles and past repair reports.
- **Dashboard and History:** A personal dashboard displays saved vehicles, repair history, and past reports for ongoing tracking.
- **Parts Pricing Transparency:** The system provides real-time parts prices from online retailers to ensure cost accuracy.

Out-of-Scope Features (Phase 1):

- Internal Diagnostics (Engine, transmission, and chassis detection) are excluded.
- Mobile Apps (Only a web-based platform will be delivered in this phase).
- Mechanical scheduling or E-commerce integration.

Use Case Example 1: A student with limited funds uploads a photo of a dented bumper. TrueRepairs identifies the part, comparing local repair shop costs with DIY options, and shows that replacing the bumper themselves would save $150.

Use Case Example 2: A parent enters their vehicle VIN to find the cost of replacing a brake pad. TrueRepairs provides both the part cost and a mechanical estimate, helping them choose whether to do it themselves or seek professional service.

By defining scope carefully, TrueRepairs delivers a practical and achievable system while leaving room for future enhancements.

1.4 Definitions, Acronyms, and Abbreviations

Term/Mnemonic	Definition
AI (Artificial Intelligence)	Computer systems capable of pattern recognition used to detect damaged car parts in uploaded photos.
API (Application Programming Interface)	A set of rules allowing different systems to communicate (e.g., retrieving repair data from OpenAI).
Cloudinary	Cloud-based media storage service used for uploading vehicle images.
DIY (Do-It-Yourself)	Refers to drivers performing their own repairs (e.g., buying and installing a replacement part).
JWT (JSON Web Token)	A compact token format used for secure session management and authentication.
KBB (Kelley Blue Book)	Competitor service focusing on vehicle valuations.
MongoDB Atlas	Cloud-hosted NoSQL database used to store user accounts, vehicles, and reports.
RepairPal	Competitor service providing "fair price" estimates without DIY comparisons.
UI (User Interface)	The visual components of the system, including the homepage, dashboard, and report screens.
VIN (Vehicle Identification Number)	A unique 17-character identifier for vehicles used by TrueRepairs for precise vehicle matching.

1.5 References

Ref.	Title/Source	Author/Publisher	Date/Access
[1]	**TrueRepairs Development Plan**	**TrueRepairs Team, Wayne State University**	**2025**
[2]	**TrueRepairs Proposal**	**TrueRepairs Team, Wayne State University**	**2025**
[3]	**OpenAI API Documentation**	**OpenAI**	**2025**
[4]	**MongoDB Atlas Documentation**	**MongoDB**	**Oct. 9, 2025**

[5]	**IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998)**	**IEEE Computer Society**	**1998**

1.6 Overview of the SRS Structure

This SRS is divided into six major sections:

- **Section 1 – Introduction:** Provides context, purpose, scope, definitions, and references.
- **Section 2 – General Factors:** Describes system perspective, functions, user characteristics, constraints, assumptions, and dependencies.
- **Section 3 – Functional Requirements:** Documents external interfaces (UI, hardware, software, communications) and detailed functional requirements (FRs).
- **Section 4 – Non-Functional Requirements:** Lists quality attributes (performance, reliability, security), design constraints, and database requirements (NFRs).
- **Section 5 – Analysis Models:** Presents data flow diagrams (DFDs) showing how data moves between users, system processes, and external APIs.
- **Section 6 – Appendix:** Includes supplemental material such as team organization and placeholders for extended diagrams/screenshots.

This structure follows IEEE standards, ensuring that each requirement is traceable and verifiable. By presenting the document in this format, the team creates a shared baseline that is both academically rigorous and practically useful for software development.

---

2. General Factors

2.1 Product Perspective

TrueRepairs is a web-based application that provides transparency in vehicle repair costs by consolidating data from multiple trusted sources. Unlike standalone desktop programs or mobile-only solutions, it is designed to operate entirely within a web browser, making it accessible across devices and operating systems.

The system is a new, independent platform. While it leverages external services to provide information, it does not extend or replace an existing product. Instead, TrueRepairs positions itself as a centralized point where car owners can obtain repair cost estimates, diagnostic insights, and part pricing information in real time.

2.1.1 Context

Traditionally, car owners had to rely on mechanics for repair quotes, often without transparency into how prices were calculated. This lack of visibility led to uncertainty and possible overcharge. TrueRepairs addresses this by combining multiple sources—such as diagnostic data, cost references, and part pricing—into a single interface that empowers users with clear, data-driven estimates.

2.1.2 System Perspective

From a high-level perspective, TrueRepairs operates as a client–server web application supported by cloud services and external data providers.

- Users access the system through standard web browsers.
- External services provide repair data, diagnostic information, and market prices.
- Cloud-hosted resources support secure data storage and system scalability.

Detailed descriptions of the system architecture, chosen frameworks, APIs, and technical integrations are beyond the scope of this SRS and are provided in the Design Document.

2.1.3 Diagram

Figure 1 provides a high-level context diagram of TrueRepairs. Users interact with the system through a browser, which connects to the platform. The platform, in turn, communicates with external data providers and cloud services to deliver repair estimates and cost comparisons.

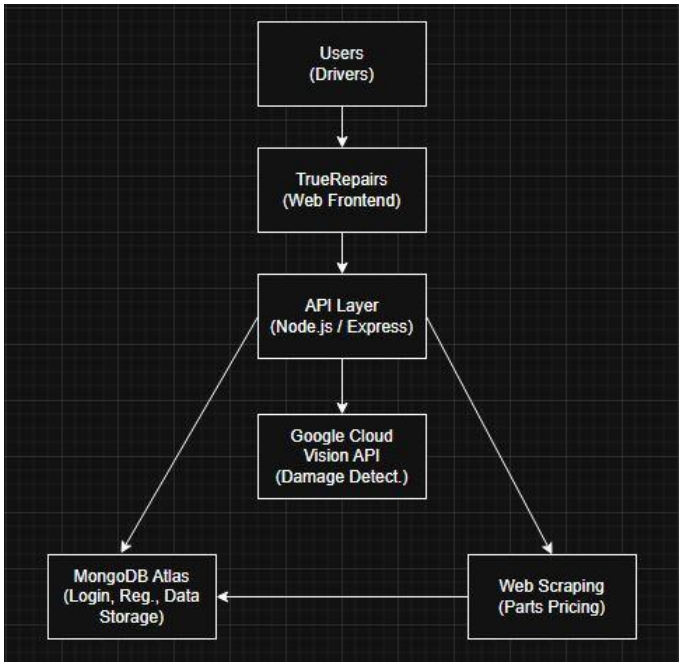


Figure 1: TrueRepairs System Architecture Diagram

2.2 Product Functions

TrueRepairs provides several core functions that are essential for meeting its objectives. These functions are not only user-facing but also supported by backend logic, APIs, and cloud services.

2.2.1 User Registration and Login

Users can create secure accounts to save vehicle information and repair history. Passwords are hashed with bcrypt, and authentication is managed using JWT tokens. MongoDB Atlas securely stores account information.

2.2.2 Repair Cost Estimation

By entering a VIN, make, model, or year, users can request a cost estimate. The system retrieves relevant data from APIs, calculates labor and parts costs, and displays the results in an easy-to-read format.

2.2.3 AI-Powered Damage Detection

Users can upload a photo of their vehicle. OpenAI identifies damaged exterior panels, such as bumpers or doors. These results are then linked to repair costs. This feature is especially useful for users unfamiliar with technical automotive terminology.

2.2.4 Dashboard and Repair History

Each registered user has access to a personalized dashboard. The dashboard displays previously saved repair estimates, uploaded images, and vehicle information. This allows users to track ongoing maintenance and compare past costs.

2.2.5 Parts Pricing Transparency

TrueRepairs leverages web scraping to gather real-time prices from automotive retailers. These prices are compared to labor-based shop estimates, giving users a transparent view of cost breakdowns.

2.3 User Characteristics

TrueRepairs is designed for a diverse user base, ranging from tech-savvy car enthusiasts to everyday drivers with little automotive knowledge. The system emphasizes **accessibility and usability** to ensure that it benefits all users equally.

2.3.1 DIY Enthusiasts

These users often perform their own repairs and value detailed cost breakdowns. They will use TrueRepairs to verify part prices and confirm whether DIY repairs are cost-effective.

2.3.2 Young Professionals

This group has limited automotive knowledge and values quick, convenient estimates. They may rely more heavily on the AI-powered damage detection feature and mobile-friendly design.

2.3.3 General Drivers

Most drivers fall into this category. They simply want fair and transparent repair estimates without needing technical expertise. For them, the system must be intuitive, straightforward, and reliable.

2.4 General Constraints

Like any system, TrueRepairs operates within several constraints:

2.4.1 Technical Standards

- Built on the MERN stack (MongoDB, Express, React, Node.js).
- Secure authentication with bcrypt and JWT.
- Compliance with HTTPS/TLS protocols for secure communication.

2.4.2 Hardware and Software Limitations

- Requires a stable internet connection; offline use is not supported.
- Performance may vary on older devices and browsers.
- Dependent on uptime of external services (MongoDB Atlas, OpenAI API).

2.4.3 Performance Constraints

- The backend must handle concurrent requests efficiently.
- AI image analysis is dependent on OpenAI's processing speed.
- System should aim for sub-second response times for queries and under 10 seconds for AI scans.

2.5 Assumptions and Dependencies

2.5.1 Assumptions

- Users will have modern browsers and internet access.
- Users will provide accurate vehicle information and images.

- Users intend to obtain estimates for guidance, not legal repair quotes.

2.5.2 Dependencies

- **MongoDB Atlas:** For accounts, vehicles, and repair history.
- **OpenAI API:** For AI-based damage detection.
- **Web Scraping:** For live parts pricing.
- **Libraries:** dotenv for configuration, bcrypt for password hashing, JWT for authentication, Axios for HTTP requests.

---

3. Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

TrueRepairs website provides multiple user interfaces that allow visitors to interact with its services. The first page that users encounter is the homepage, which serves as the entry point to the system. The theme colors of the homepage are blue and white. At the top of the homepage, a white navigation bar is displayed, containing the site logo and navigation links to the Features, About Us, and Contact sections. These links scroll the user directly to the corresponding section on the same page. The navigation bar also provides two buttons: Sign In for returning users and register for new visitors. Both buttons open popup boxes where users can either log in with their credentials or create a new account.

The body of the homepage highlights the core content of the website. At the top, a header section displays an image of a car with text introducing the platform, along with a prominent Get Started button; clicking this button opens the registration popup for quick access. Below the header, the homepage uses a white background and presents content in a card-style layout. Each major section (Features, About Us, Contact) is represented by a content card that typically includes an image that visually represents the section placed next to a gray text container providing supporting information. This gray panel provides visual separation and improves text readability against the white background. Cards may alternate image/text order on larger screens and stack vertically on smaller screens to preserve readability and usability. At the bottom of the homepage, a blue footer is displayed, containing quick navigation links, the site logo, and copyright information. This section ensures consistency across the interface and allows users to easily navigate or recognize the brand regardless of where they are on the site.

The second user interface is the dashboard page, which appears after a user registers or signs in. The dashboard includes a white navigation bar with the site logo and a Sign Out button, ensuring consistency with the homepage. The body of the dashboard features a light gray background with box-shaped white containers that display clickable icons for different services. These include Vehicle Information, Profile, AI Scan, Settings, Reports, and Support. This card-style layout helps the icons stand out clearly while maintaining a clean, modern design. Blue is used as the primary highlight color for buttons and interactive elements. The footer remains consistent with the homepage, providing uniformity across the application.

3.1.2 Hardware Interfaces

To use the TrueRepairs software product, a desktop or laptop computer is the recommended hardware for optimal performance in terms of speed, stability, and visual display. The operating system must be Windows 10 or later, or macOS High Sierra (version 10.14) or later. The system should include at least an Intel Core I5 processor. Since TrueRepairs depends on user input to generate accurate results, standard input devices are required. For desktops, a USB 3.0 or Bluetooth-connected keyboard and mouse are necessary to capture input, while laptops may use the integrated keyboard and touchpad.

Alternative hardware such as tablets and smartphones can also be used to access TrueRepairs, though these devices may experience reduced performance due to smaller screens and lower processing power. The operating system must be Android 9.0 or newer, or iOS 10.0 or newer. On such devices, inputs and outputs are handled through a capacitive touchscreen, with user interactions performed via tapping, swiping, and scrolling. TrueRepairs employs responsive design principles to dynamically adjust its user interface to fit varying screen dimensions and maintain usability across devices.

Alternative hardware such as tablets and smartphones can also be used to access TrueRepairs, though these devices may experience reduced performance due to smaller screens and lower processing power. The operating system must be Android 9.0 or newer, or IOS 10 or newer. Mobile and tablet devices should include at least an Apple A9 processor for iOS, with a Qualcomm Snapdragon 600 series or higher recommended for Android. On such devices, inputs and outputs are handled through a capacitive touchscreen, with user interactions performed via tapping, swiping, and scrolling. TrueRepairs employs responsive design principles to dynamically adjust its user interface to fit varying screen dimensions and maintain usability across devices.

3.1.3 Software Interfaces

TrueRepairs integrates multiple software products and interfaces to ensure efficient communication and interaction between its components, including the frontend, backend, and external APIs:

- MongoDB Atlas (Mnemonic: MongoDB) is a cloud-hosted service that currently uses the stable Version 8.0.13 release, provided by MongoDB Inc. Our system interacts with this interface by utilizing its data management functionality to store user data such as account information, vehicle details, and repair reports. Communication with MongoDB is handled in the backend through Node.js and Express.js, using JSON-based queries and responses
- Mongoose (Mnemonic: Mongoose) is a schema-based solution for modeling application data. The version in use is 7.6.0, and it is provided by MongooseJS. Our system depends on this interface as an Object Data Modeling (ODM) library for MongoDB Atlas, where it defines schema rules for TrueRepairs' stored data to ensure data consistency.
- Cloudinary (Mnemonic: Cloudinary) is a cloud-based media management service. The version in use is N/A, and it is provided by Cloudinary Inc. This interface serves TrueRepairs as the database for storing images uploaded during the AI Scan, which are later retrieved from Cloudinary to be included in the repair report. The communication process works through URLs: Cloudinary generates a unique URL for each image, and that URL is embedded in the report to retrieve and display the image.
- OpenAI API, also known as ChatGPT is a cloud-based image recognition service used to detect and label car parts for accurate repair report generation. Communication occurs via HTTPS POST requests containing JSON with an image URL, and the service returns JSON with detected objects and labels. The version in use is v1, and it is provided by OpenAI.

3.1.4 Communication Interfaces

TrueRepairs uses standard web protocols to ensure reliable and secure data transfer between the frontend, backend, and external services. The backend is built on Node.js and Express.js, and all communication occurs over HTTPS Port 443 to maintain security and integrity. External API requests are also made via HTTPS, with user inputs sent as JSON requests and responses returned in JSON format. This approach ensures that data remains structured, efficient, and protected against interception or unauthorized access. Frontend runs locally via https port 5137 while as the backend runs via port 3000.

3.1.5 Functional Requirements

ID	FR0.1
Title	User Registration
Description	Allow new users to create accounts with a username, email, and password.
Input	Username, email, password, confirm password
Processing	Validate password strength, confirm email uniqueness, hash password, store in database.
Outputs	Confirmation message and new user account created.
Error Handling	Display error messages when password does not meet strength requirements: • Display error message: “At least 8 characters long.” • Display error message: “At least 1 lowercase letter (a-z)” • Display error message: “At least 1 uppercase letter (A-Z)” • Display error message: “At least 1 number (0-9)” • Display error message: “At least 1 special character (@ # $ % ^ &)”
Dependencies	MongoDB Atlas
Priority	High

ID	FR0.2
Title	User Sign-In
Description	Allow existing users to access their accounts.
Input	Email/username, password.
Processing	Verify credentials against stored data in MongoDB.
Outputs	Access to user dashboard.
Error Handling	Error messages for invalid credentials or unregistered emails. • Display error message for invalid credentials: “The password is incorrect. Please try again.” • Display error message for unregistered emails: “User does not exist, Please try again.”

Dependencies	FR1
Priority	High

ID	FR0.3
Title	Profile Management
Description	Enables users to update account details such as email, and password.
Input	New email or password.
Processing	Validate input and update the database.
Outputs	Confirmation of changes
Error Handling	Error message for incorrect current password or emails already exist. • Display error message for incorrect password: “Incorrect password. Please try again.” • Display error message for already registered email: “That email is already in use.”
Dependencies	FR02
Priority	Medium

ID	FR0.4
Title	Settings Managment
Description	Allows users to deactivate or delete their TrueRepairs account.
Input	User-selected deactivate or delete account button.
Processing	The system deactivates or removes account.
Outputs	Confirmation of changes.
Error Handling	Display error message for incorrect password for confirmation. • Display error message: “Incorrect password:
Dependencies	FR02
Priority	Medium

ID	FR0.5
Title	AI Scan Diagnostic
Description	Uses OpenAI to detect damaged car parts from uploaded images.
Input	Uploaded vehicle image.
Processing	Validate image format; send to OpenAI; map detected parts to repair data.
Outputs	Labels of damaged parts.
Error Handling	Messages for unsupported formats, processing failures, or no provided image: • Display error message for no provided image: “At least one image is required”.
Dependencies	FR02
Priority	High

ID	FR0.6
Title	Automated Repair Report Generation
Description	Generate repair report combining AI Scan results and vehicle info
Input	Make, Model, Year, uploaded Image
Outputs	Detailed repair report
Process	Retrieve the uploaded image from Cloudinary Retrieve labels from AI Scan Diagnostic Retrieve vehicle information from the Vehicle profile Retrieve repair costs from the OpenAI API Generate the repair report Store the completed report in the database
Error Handling	Display an error message if the report cannot be generated due to failure in dependent sources, such as AI scan, OpenAI API.
Dependencies	Forward: None Backward: FR0.2, FR0.3, FR0.5, FR0.7
Priority	High

ID	FR0.7
Title	Vehicle Information Profile
Description	Allows the user to create vehicle profiles that hold user vehicle information.
Input	Model, Year, Make, VIN
Processing	Validate user input and vehicle details using VehicleDatabases API. Save the vehicle information in database.
Outputs	Vehicle profile created
Error Handling	Error messages are displayed in empty fields, invalid input.
Dependencies	Forward: FR0.6 Backward: FR0.2
Priority	High

ID	FR0.8
Title	Report Management
Description	Manage repair reports: view, save, delete
Input	Clickable icons: report, delete, save icons
Processing	Load and manages report based on user three actions
Outputs	Report displayed or deleted
Error Handing	Error message display for failed action
Dependencies	Forward: FR0.6 Backward: FR0.2 FR.05 FR0.6 FR0.7
Priority	Medium

3.3 Non-Functional Requirements

3.3.1 Performance

- The system shall process at least 80% of repair cost requests in under 2 seconds.
- AI damage detection shall return results in under 10 seconds for images < 5 MB.
- The front end shall render all pages, including charts, in under 1.5 seconds on average consumer hardware.

3.3.2 Reliability

- The system should always maintain data consistency and integrity.
- MongoDB shall acknowledge all database writes.

- The system shall log errors and recover gracefully.

3.3.3 Availability

- System uptime shall be at least 99.5% per month, excluding scheduled maintenance.
- API failures (OpenAI) shall not crash the app; fallback data or messages shall be used.
- User authentication and report viewing remain available even if one API is down.

3.3.4 Security

- Passwords hashed with bcrypt; plaintext storage prohibited.
- All API calls use HTTPS/TLS.
- Sensitive operations logged for auditing.
- Access control limits users to their own data; future admin accounts have elevated privileges.

3.3.5 Maintainability

- Codebase shall be modular (frontend, backend, API handlers, AI).
- Documentation shall cover endpoints, database schemas, and workflows.
- Dependencies shall follow versioning standards to ease updates.

3.3.6 Portability

- Application supports all modern browsers (Chrome, Firefox, Safari, Edge).
- Backend deployable on Node.js servers or via Docker.
- Services chosen to minimize vendor lock-in (MongoDB Atlas, Cloudinary, OpenAI).

3.4 Design Constraints

3.4.1 Hardware Limitations

System hosted in the cloud; users access via browsers. Performance must account for a range of devices, from laptops to smartphones. Image uploads optimized to reduce bandwidth.

3.4.2 API Dependencies

Relies on OpenAI API. Constraints include API rate limits, latency, and contract changes. System must fail gracefully with fallback messages.

3.4.3 Standards Compliance

- JWT authentication for security.

- Bcrypt for password storage.
- Compliance with common web security practices.

3.4.4 Client Constraints

Reports must present data clearly for everyday drivers without requiring prior automotive knowledge.

3.5 Logical Database Requirements

3.5.1 Database Usage

MongoDB Atlas stores all user, vehicle, and repair report data. Cloudinary stores images, with URLs referenced in MongoDB.

3.5.2 Types of Information

- Users: firstName, lastName, email (unique), hashed password.
- Vehicles: VIN, make, model, year, userId reference.
- Repair Reports: vehicleId reference, parts, costs (DIY/professional), AI data, timestamps.

3.5.3 Frequency of Use

- High: login/logout, report generation.
- Medium: image uploads, profile updates.
- Low: backups, admin queries, audit logs.

3.5.4 Accessing Capabilities

- Users: read/write own data.
- Admin (future): global access.
- JWT tokens enforce permissions.

3.5.5 Data Entities and Relationships

- User → Vehicle (1:M).
- Vehicle → Repair Report (1:M).
- Repair Report → Image(s) (optional, 1:M).

3.5.6 Integrity Constraints

- Unique emails required.

- Required fields validated for type/format.
- Foreign key references enforced.
- Passwords always hashed.

3.5.7 Data Retention

- Accounts and reports remained until deletion was requested.
- Audit logs remained for 90 days.
- Images follow user deletion/report removal rules.

3.6 Other Requirements

At this stage, no additional requirements are identified beyond those covered above.

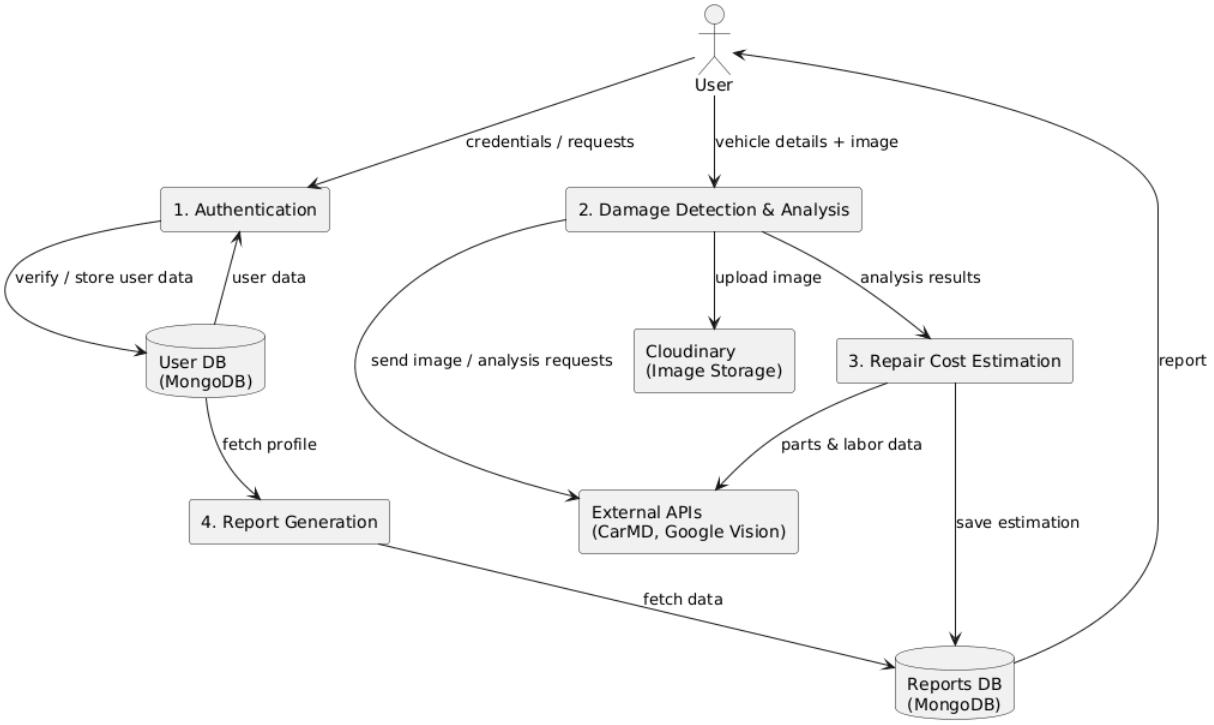
3.7 Consolidated Error Handling

Although each functional requirement specifies its own error handling, the system employs a consistent, centralized approach:

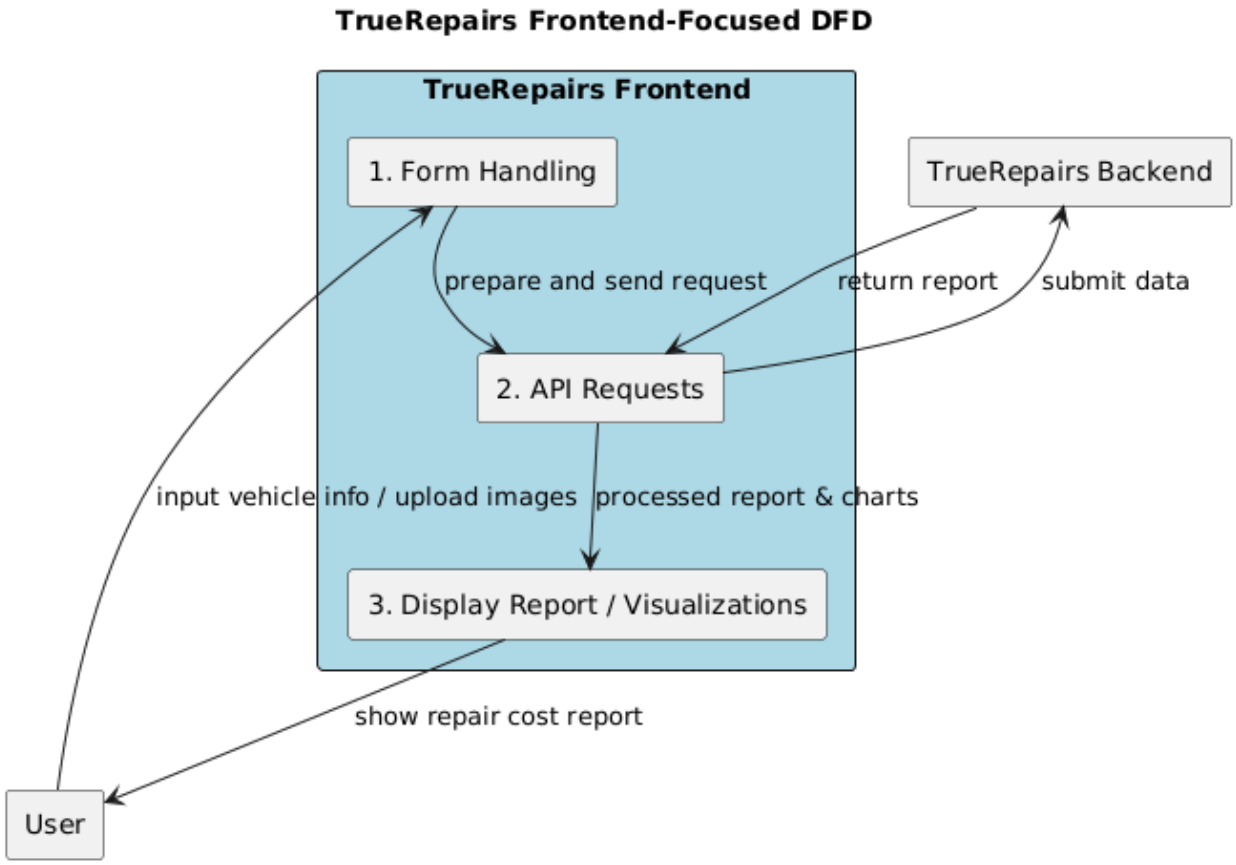
- **Authentication Errors**
  Invalid or expired credentials return clear, non-technical messages. Rate limiting prevents brute-force attempts.
- **API Failures**
  If external APIs (OpenAI API) fail or timeout, the system displays fallback messages ("Data unavailable, please retry") while logging the incident for developers.
- **Image Upload Issues**
  Uploading unsupported formats or oversized files triggers descriptive alerts. Only JPEG and PNG (≤5 MB) are accepted.
- **Database Failures**
  Failures in read/write operations display a generic "System error" to users while capturing logs internally.
- **Form Validation**
  Client-side and server-side validation ensures integrity. Invalid VINs, missing fields, or unsafe characters result in immediate error prompts.

---

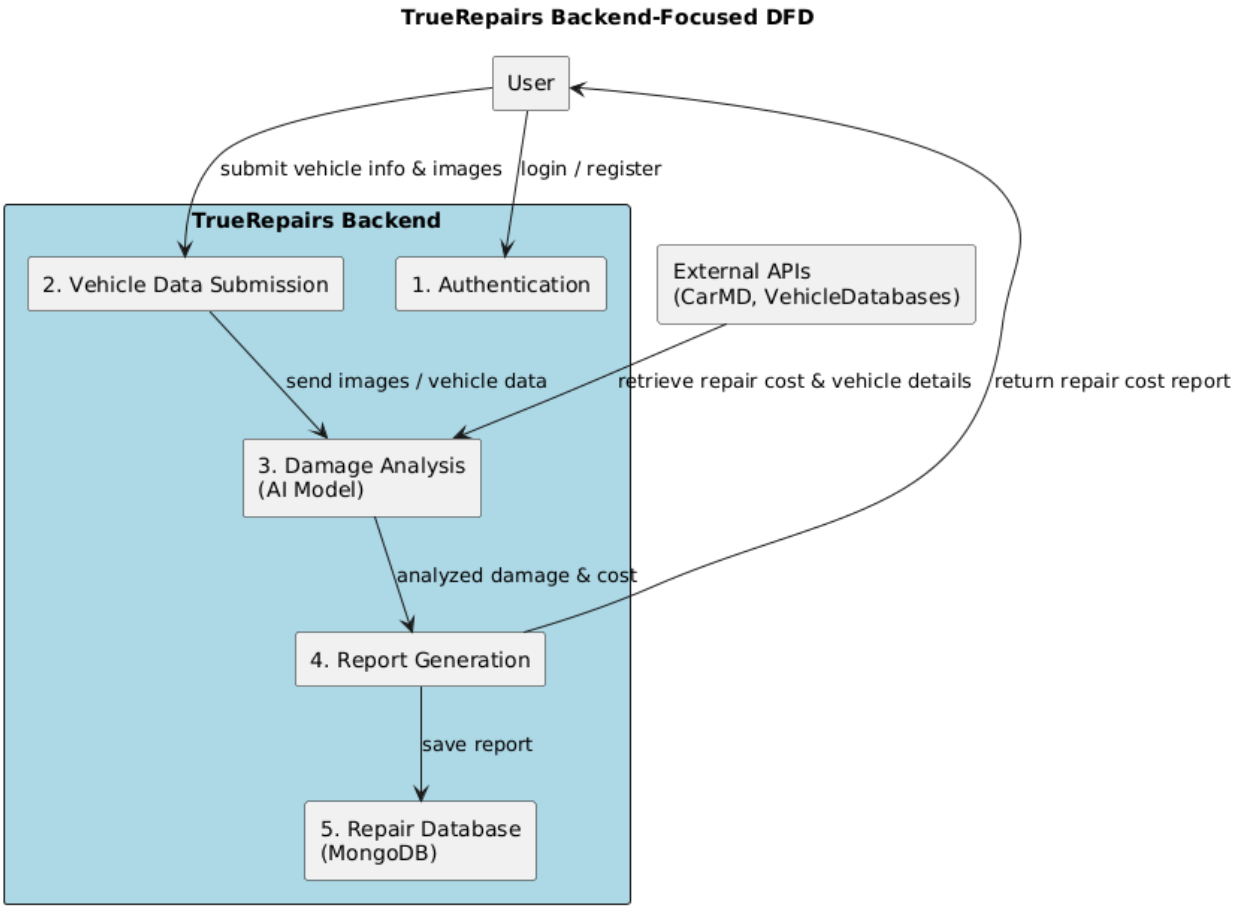
4. Analysis Models



The diagram above shows the data flow between components in the TrueRepairs application. The process begins with the user, who provides login credentials and vehicle details, including photos. The system verifies credentials against the user database before allowing access. Vehicle images and details are passed to the damage detection and analysis process, where an AI model evaluates the extent of repairs needed. Results from this analysis are stored in the repair database and used to generate a report. The report is then returned to the user through the application. Dashed lines represent the boundaries between external entities (users, databases) and the TrueRepairs system itself.



The diagram above shows the data flow between components in the TrueRepairs frontend application, which serves as the user interface. The data originates from the user when they input vehicle information or upload images. The frontend handles form validation and prepares API requests to submit this data to the backend server. Once the backend processes the information and returns a repair report, the front end receives it, generates visualizations such as charts, and displays the results to the user. Dashed lines represent the system boundary between the frontend and backend, with all communication occurring via API calls over the network.



The diagram above shows the data flow between components in the TrueRepairs backend system. User inputs, such as vehicle information and uploaded images, enter the system and are processed by modules for authentication, vehicle data handling, and damage analysis using AI models. External API like OpenAI and Vehicle Databases, provide additional repair cost and vehicle information to support analysis. The processed data is stored in the MongoDB repair database and used to generate a comprehensive repair report, which is then returned to the user. Dashed lines represent the system boundary between the backend and external entities, showing how data flows through the system for processing, storage, and reporting.

---

5. Traceability Matrix

Requirement ID	Source (User Need / Goal)	Related Implementation	Priority
FR1 – Registration	Secure user account creation	User schema, bcrypt hashing	High
FR2 – Login	Access to saved reports & dashboard	JWT authentication, sessions	High
FR3 – Profile Mgmt.	Ability to manage account details	Account settings module	Medium
FR5 – AI Scan	Identify vehicle damage visually	OpenAI API integration	High
FR6 – Report Gen	Transparent repair comparison	Scraping module + filtering	High
FR7 – Vehicle Info	Store vehicles for repeat use	MongoDB vehicle collection	High
FR8 – Report Mgmt.	Save and delete reports	MongoDB + dashboard integration	Medium
NFR – Performance	Fast responses & AI results	Backend concurrency, caching	Critical
NFR – Security	Protect user data	HTTPS, bcrypt, JWT, TLS	Critical
NFR – Availability	Reliability for users	Cloud hosting, fallback handling	High